

Local SecOps AI Agent

Sovereign Workspace

1 Project Overview

The advent of powerful AI coding assistants has introduced unprecedented agentic capabilities to modern engineering workflows, enabling rapid codebase analysis, terminal execution, and automated file manipulation. However, within strict SecOps, Blue Teaming, and Zero-Trust environments, these tools present a critical attack surface. Sending proprietary infrastructure-as-code (IaC), sensitive network topologies, or raw security logs to cloud-based LLM APIs constitutes a massive data exfiltration risk and violates fundamental compliance mandates.

The **Local SecOps AI Agent (Sovereign Workspace)** project engineers a solution to this security dilemma. By replicating cloud-dependent AI agent capabilities entirely on-premises, this project delivers a 100% air-gapped, sovereign AI agent workspace. This architecture guarantees that all retrieval-augmented generation (RAG), context processing, and agentic actions (reading/writing files, running terminal commands) occur locally, ensuring zero bytes of sensitive telemetry or intellectual property leave the host machine.

2 Architecture & Tech Stack

The Sovereign Workspace relies on a decoupled architecture, separating the inference engine from the user interfaces (IDE and CLI).

- **LLM Engine:** Ollama running locally on dedicated GPU hardware. This serves as the local inference endpoint. **Crucially, we utilize models that natively support Tool Calling (Function Calling)** (e.g., llama3.1, qwen2.5) so the agentic harness can autonomously execute actions like reading/writing files and running commands.
- **IDE Integration:** Continue.dev extension for VS Code. This replicates the cloud-based IDE assistant experience, providing local models with full RAG context over the local file system.
- **CLI Integration:** Pi Agent. This replicates cloud-based CLI agent capabilities, allowing for terminal-native interactions directly connected to the local Ollama instance, configured entirely via local JSON files.

3 Implementation Guide: Step-by-Step Setup

3.1 Prerequisites & Hardware

To achieve acceptable inference speeds for RAG and tool-calling, the host machine requires adequate hardware acceleration:

- **GPU:** Dedicated GPU with at least 8GB VRAM (e.g., NVIDIA RTX 3060/4060 or better) for smaller models (8B), or 16GB+ VRAM for larger context windows. Apple Silicon (M1/M2/M3 Max/Ultra) provides excellent unified memory performance.

- **RAM:** 32GB system RAM minimum (64GB recommended).
- **OS:** Linux (Ubuntu 22.04+ / Debian 12+), macOS, or Windows 10/11 (native or via WSL2).

3.2 Step 1: The Engine (Ollama Setup)

Ollama serves as the local inference endpoint, providing an OpenAI-compatible API over `localhost:11434`.

Installation (Linux/macOS):

```
curl -fsSL https://ollama.com/install.sh | sh
```

(For Windows, download the installer from ollama.com).

Pulling Tool-Calling Models: For the Pi agent to take actions (read, write, execute), the model **must** support tool calling. We use `llama3.1` (or equivalent tool-capable models). We also pull an embedding model for RAG.

```
# Pull a tool-calling capable model for the agent harness
ollama pull llama3.1

# Pull the embedding model for local RAG (IDE)
ollama pull nomic-embed-text
```

3.3 Step 2: The IDE Integration (Continue.dev)

1. Open VS Code, navigate to Extensions (`Ctrl+Shift+X`), and install **Continue**.
2. Open the Continue configuration file at `~/.continue/config.json`.
3. Replace the contents with this strict local-only configuration:

```
{
  "models": [
    {
      "title": "Local Llama 3.1 (Tool-Capable)",
      "provider": "ollama",
      "model": "llama3.1",
      "apiBase": "http://localhost:11434"
    }
  ],
  "tabAutocompleteModel": {
    "title": "Starcode2 (Local Autocomplete)",
    "provider": "ollama",
    "model": "starcode2:3b",
    "apiBase": "http://localhost:11434"
  },
  "embeddingsProvider": {
    "provider": "ollama",
    "model": "nomic-embed-text",
    "apiBase": "http://localhost:11434"
  },
  "allowAnonymousTelemetry": false
}
```

Note: Run `ollama pull starcode2:3b` if using the autocomplete feature.

3.4 Step 3: The CLI Agent (Pi Integration via JSON Files)

We configure the Pi agent using its native JSON configuration files to maintain a clean, file-based setup. This allows us to explicitly define tool-calling capabilities.

1. Installation:

```
pip install pi-agent
```

2. Configuration (settings.json): Locate or create the Pi agent's `settings.json` file (typically in `~/.pi/` or the app's config directory). Configure it to point the API to localhost.

```
{
  "api_base": "http://localhost:11434/v1",
  "api_key": "ollama",
  "default_model": "llama3.1",
  "max_tokens": 4096,
  "temperature": 0.2
}
```

3. Model Definition (models.json): Ensure your `models.json` explicitly maps the model and confirms tool-calling support (`"supports_tools": true`). This is what lets the agent harness know it has permission to execute file I/O and terminal actions.

```
{
  "llama3.1": {
    "provider": "ollama",
    "context_window": 8192,
    "supports_tools": true,
    "description": "Tool-capable local model for agentic actions (Read, Write, Execute).",
  }
}
```

4. Execution Examples:

You can now use the Pi agent to autonomously analyze logs or execute scripts. Because the model is tool-capable, it can read files or run commands dynamically based on your prompt.

Linux / macOS Example:

```
# The agent will use its read/write tools to process the log and create a script
pi "Read /var/log/auth.log, identify the top 3 IPs failing SSH authentication, and write an iptables block script to block_ips.sh."
```

Windows (PowerShell) Example:

```
# The agent will use its tools to read Event Logs and generate a mitigation script
pi "Read the last 50 Security Event Logs for ID 4625. Identify the source IPs and write a PowerShell script to block them in Windows Firewall."
```

4 Security & Privacy Impact

The implementation of this Local SecOps AI Agent yields a massive Return on Investment (ROI):

- **Zero-Trust Compliance:** Eliminates reliance on external APIs. The attack surface associated with third-party data processors is eradicated.
- **Zero Risk of Data Leakage:** 100% of telemetry, source code, network typologies, and security logs remain on-premises.

- **Autonomous Agentic Workflows:** Because the local models (llama3.1) explicitly support function/tool calling, the agent acts autonomously (reading/writing files, executing scripts) without breaking the air gap.
- **Zero API Costs:** Infinite scalability within the local hardware constraints.